

Documentación Técnica — Framework de Testing de API con Playwright + TypeScript + Zod

Documentación de referencia del diseño, las decisiones técnicas y el funcionamiento del proyecto, incluyendo las alternativas evaluadas con sus ventajas y desventajas.

Índice

1. Alcance
2. Qué es el testing de API y por qué importa
3. La API bajo prueba: Restful-Booker
4. Por qué Playwright para API (vs Postman vs otras librerías)
5. Contract testing y por qué Zod
6. Anatomía del proyecto
7. API Clients: el Page Object del mundo API
8. Schemas: contratos y fuente de verdad de los tipos
9. Fixtures de API
10. Setup por API: el concepto que faltaba
11. Datos con Builder: deep clone vs shallow clone
12. Autenticación: token, Cookie y 403
13. Métodos HTTP, códigos de estado e idempotencia
14. Encadenamiento: el CRUD end-to-end
15. Casos negativos: probar lo que NO debe pasar
16. Aislamiento en una API compartida
17. La configuración de Playwright para API
18. CI/CD para API (más liviano)
19. Extensiones sugeridas
20. Glosario
21. Próximos pasos

1. Alcance

Este es el **Proyecto 2** de la serie. En el Proyecto 1 automatizamos la **interfaz** (UI) de una tienda. Acá automatizamos la **API**: probamos el backend directamente, sin navegador, haciendo llamadas HTTP y verificando las respuestas.

Muchos conceptos se comparten con el Proyecto 1 (fixtures, builders, config por ambiente, etiquetas @smoke/@regression). Eso es **a propósito**: un buen framework mantiene coherencia. Lo nuevo acá es el **contract testing**, los **API Clients**, la **autenticación** y el **encadenamiento** de requests.

La API bajo prueba es **Restful-Booker**, un sistema de reservas de hotel de demostración, público y ampliamente utilizado como banco de pruebas para testing de API.

2. Qué es el testing de API y por qué importa

Una **API** (Application Programming Interface) es la puerta de entrada al backend: recibe requests HTTP (con un método, una URL, headers y a veces un body) y devuelve respuestas (un código de estado y, normalmente, un body en JSON). El frontend que probamos en el Proyecto 1, por debajo, no hace otra cosa que llamar a una API como esta.

Volviendo a la pirámide



El testing de API está en la **capa del medio**, y tiene ventajas enormes sobre el de UI:

	Testing de UI (Proyecto 1)	Testing de API (Proyecto 2)
Velocidad	Segundos por test	Milisegundos por test
Estabilidad	Sensible a cambios visuales	Solo cambia si cambia el contrato
Qué prueba	Todo el stack integrado	La lógica de negocio del backend
Dependencias	Navegador, render, selectores	Solo HTTP
Flakiness	Más propenso	Mucho menos propenso

Por eso la estrategia sana es cargar la mayor parte de la cobertura en la capa de API y reservar la UI para los pocos flujos que de verdad hay que ver en pantalla. Un bug de cálculo de precio se prueba mucho mejor (más rápido, más estable, con más casos) pegándole a la API que navegando la UI.

Números de este proyecto

12 tests corren en ~2 segundos. Compará con el Proyecto 1: 39 ejecuciones en ~10-20 segundos. Y acá ni siquiera instalamos un navegador. Esa es la eficiencia de la capa de API.

3. La API bajo prueba: Restful-Booker

Restful-Booker expone endpoints para gestionar reservas. Los que usamos:

Método	Endpoint	Qué hace	¿Auth?
GET	/ping	Health check (¿está viva?)	No
POST	/auth	Devuelve un token	No
GET	/booking	Lista de IDs de reservas	No
GET	/booking/{id}	Detalle de una reserva	No
POST	/booking	Crea una reserva	No
PUT	/booking/{id}	Reemplaza una reserva completa	Sí
PATCH	/booking/{id}	Actualiza campos parciales	Sí
DELETE	/booking/{id}	Elimina una reserva	Sí

Las "rarezas" que descubrimos verificando (no asumiendo)

Antes de escribir un solo test, hicimos **reconocimiento** de la API con `curl`. Esto es clave: un QA de API **nunca asume** cómo se comporta un endpoint, lo verifica. Y encontramos tres comportamientos que no son los "de manual":

1. **Auth inválida devuelve 200, no 401**. Con credenciales incorrectas, la API responde `200 OK` con un body `{ "reason": "Bad credentials" }`. Si hubiéramos asumido que un login fallido da 401, el test sería incorrecto.
2. **DELETE devuelve 201 Created, no 200 / 204**. Es un comportamiento no estándar (borrar y responder "Created" es raro), pero es el real. Nuestro test verifica `201`.
3. **La autenticación va por header Cookie: token=...**. No es el típico `Authorization: Bearer`. Sin ese header, los endpoints protegidos devuelven `403 Forbidden`.

4. Por qué Playwright para API (vs Postman vs otras librerías)

Elegimos el **testing de API de Playwright** (el fixture `request` / `APIRequestContext`). Alternativas y por qué:

Herramienta	Qué es	Pros	Contras
Playwright <code>request</code>	Cliente HTTP del runner de Playwright	Mismo stack que la UI; TS nativo; corre en el mismo CI; sin navegador	Menos "famoso" para API que otros
Postman / Newman	App gráfica + runner CLI	Rápido para explorar; visual	Los tests viven en JSON/GUI, difícil de versionar y revisar como código; escala mal
REST Assured	Librería Java	Muy potente; estándar en mundo Java	Requiere Java; otro stack
supertest / axios + Jest	Librerías Node	Livianas y flexibles	Hay que armar más a mano (config, reportes)

Por qué Playwright acá

1. **Un solo stack para UI y API.** El Proyecto 1 y el 2 usan la misma herramienta, el mismo lenguaje, la misma forma de fixtures y reportes. Un equipo aprende una cosa, no cinco.
2. **Los tests son código versionable.** A diferencia de Postman (donde los tests viven en colecciones JSON difíciles de revisar en un pull request), acá son archivos `.ts` que pasan por code review como cualquier código.
3. `request` **sin navegador.** El fixture hace HTTP puro. Por eso el CI es más liviano y no instalamos browsers.
4. **Reportes y trace** integrados, igual que en UI.

Postman no está "mal": es excelente para **explorar** una API manualmente (de hecho, para el reconocimiento inicial es ideal). Pero para una **suite automatizada, versionada y en CI**, tener los tests como código es muy superior. Un uso común y sano: explorar con Postman/curl, automatizar con código.

5. Contract testing y por qué Zod

Este es el **concepto más importante del proyecto**, así que va despacio.

El problema: un `200 OK` no garantiza nada

Un error de principiante en testing de API es verificar solo el código de estado:

```
// ❌ Insuficiente: "responde 200" no dice si la respuesta sirve
expect(response.status()).toBe(200);
```

El status `200` solo dice "la request no explotó". Pero la respuesta podría venir con un campo faltante, un tipo cambiado (`totalprice` como string en vez de número), o una estructura distinta. Tu app se rompería y el test seguiría en verde.

La solución: validar el CONTRATO (la forma de la respuesta)

Contract testing es verificar que la respuesta tenga la **forma** acordada: qué campos, de qué tipos, qué es obligatorio y qué opcional. Para eso usamos **Zod**.

Un **schema de Zod** describe esa forma:

```
export const BookingSchema = z.object({
  firstname: z.string(),
  lastname: z.string(),
  totalprice: z.number(),           // ← si la API manda "150" (string), esto FALLA
  depositpaid: z.boolean(),
  bookingdates: z.object({ checkin: z.string(), checkout: z.string() }),
  additionalneeds: z.string().optional(), // campo opcional
});
```

Y en el test lo aplicamos con `.parse()`:

```
const body = BookingSchema.parse(await response.json());
```

Si la respuesta **no** cumple el contrato (falta un campo, cambió un tipo), `parse()` **lanza un error** con un mensaje claro que dice exactamente qué campo falló, y el test se pone en rojo. Esto detecta **cambios de contrato** (breaking changes de la API) que un `status === 200` jamás vería.

Por qué Zod y no otras opciones

Alternativa	Contra
Zod	(elegida) validación en runtime + tipos derivados en un solo lugar
Chequear campo por campo a mano	Verboso, incompleto, fácil de olvidar un campo
JSON Schema (ajv)	Válido, pero el schema es JSON separado del código y no da tipos TS
Solo tipos de TypeScript	Los tipos no existen en runtime : no validan la respuesta real, solo ayudan al escribir

El punto sutil pero poderoso: **los tipos de TypeScript desaparecen al compilar**. No pueden validar lo que la API devuelve de verdad en tiempo de ejecución. Zod sí valida en runtime **y además** genera el tipo TS. Ver la próxima sección.

6. Anatomía del proyecto

```
src/  
├─ config/      → CÓMO se conecta (URL de la API, credenciales)  
├─ schemas/     → los CONTRATOS (Zod): forma de las respuestas + tipos  
├─ clients/     → CÓMO se habla con cada grupo de endpoints (API Clients)  
├─ data/        → QUÉ datos enviamos (Builder de reservas)  
└─ fixtures/    → el PEGAMENTO que inyecta clients, token y datos  
  
tests/         → QUÉ verificamos, agrupado por recurso (auth, booking, health)
```

La lógica es la misma del Proyecto 1: **separación de responsabilidades**. Cada carpeta tiene un único motivo para cambiar. Si cambia la forma de una respuesta, tocás `schemas/`. Si cambia una URL, tocás `clients/`. Los tests quedan estables.

7. API Clients: el Page Object del mundo API

Un **API Client** es al testing de API lo que un Page Object es al de UI: **encapsula cómo se habla con un grupo de endpoints**, para que el test hable de negocio y no de detalles HTTP.

```
export class BookingClient {
  constructor(private readonly request: APIRequestContext) {}

  getById(id: number): Promise<APIResponse> {
    return this.request.get(`/booking/${id}`);
  }

  create(payload: Booking): Promise<APIResponse> {
    return this.request.post('/booking', { data: payload });
  }

  update(id: number, payload: Booking, token: string): Promise<APIResponse> {
    return this.request.put(`/booking/${id}`, {
      data: payload,
      headers: { Cookie: `token=${token}` }, // el detalle de auth vive acá
    });
  }
}
```

Lo que se gana:

- El test dice `bookingClient.update(id, datos, token)`, no arma la request HTTP a mano cada vez.
- El detalle de **cómo** se autentica (el header `Cookie`) vive en un solo lugar. Si la API cambiara a `Authorization: Bearer`, se corrige en el client y **ningún test se toca**.
- Las rutas (`/booking/{id}`) están centralizadas.

Decisión de diseño — devolver `APIResponse`, no el body ya parseado. Cada método devuelve la respuesta cruda. ¿Por qué no devolver directamente el JSON? Porque el test necesita assertar **dos cosas**: el **status code** (`expect(res.status()).toBe(200)`) y el **body** (validado con el schema). Si el client parseara el body, perderíamos el acceso al status. Devolver `APIResponse` le da al test control total.

8. Schemas: contratos y fuente de verdad de los tipos

Los schemas de Zod hacen **doble función**, y esto es elegante:

Función 1 — validación en runtime (ya vista)

```
const body = BookingSchema.parse(await response.json()); // valida la forma real
```

Función 2 — generar el tipo TypeScript

```
export type Booking = z.infer<typeof BookingSchema>;
```

`z.infer` **deriva** el tipo TypeScript directamente del schema. Así, el tipo `Booking` que usan el `BookingClient` y el `BookingBuilder` sale del **mismo lugar** que la validación. No los definimos dos veces (una para el tipo y otra para validar): **una sola fuente de verdad**.

Por qué esto importa: si mañana la API agrega un campo obligatorio, actualizás el schema **una vez**, y automáticamente: (a) la validación lo exige en runtime, y (b) el tipo TS lo refleja y el compilador te marca dónde falta. Sin `z.infer`, tendrías que acordarte de actualizar el tipo y el validador por separado, y tarde o temprano se desincronizan.

Este patrón —**schema-first**, donde el schema genera los tipos— es una práctica muy valorada en proyectos TypeScript modernos, no solo en testing.

9. Fixtures de API

Igual que en el Proyecto 1, extendemos el `test` de Playwright para inyectar lo que los tests necesitan. Acá inyectamos clients, un token y una reserva ya creada:

```
export const test = base.extend<ApiFixtures>({
  authClient: async ({ request }, use) => { await use(new AuthClient(request)); },
  bookingClient: async ({ request }, use) => { await use(new BookingClient(request)); },
  authToken: async ({ authClient }, use) => {
    const token = await authClient.getToken(ENV.username, ENV.password);
    await use(token);
  },
  createdBooking: async ({ bookingClient, authToken }, use) => {
    // ...crea una reserva por API (ver sección 10)
  },
});
```

El fixture `request` es de Playwright (no lo definimos nosotros): es el `APIRequestContext`, ya configurado con la `baseURL` y los headers por defecto de `playwright.config.ts`. Por eso los clients hacen `this.request.get('/booking')` con rutas relativas.

Composición de fixtures: notá que `authToken` depende de `authClient`, y `createdBooking` depende de `bookingClient` y `authToken`. Playwright resuelve esa cadena de dependencias automáticamente: si un test pide `createdBooking`, Playwright construye primero el client y el token, y recién después crea la reserva.

10. Setup por API: el concepto que faltaba

En el Proyecto 1 dijimos que SauceDemo no tenía backend, así que el "setup por API" quedaba pendiente. Aquí está.

El problema: un test que actualiza o borra una reserva necesita que **exista una reserva** antes de empezar. ¿Cómo la preparamos? Tres opciones:

Estrategia	Contra
Usar una reserva que "ya esté" en el server	Frágil: en un server compartido, otro puede borrarla o modificarla
Crearla navegando la UI	Lento y depende de que la UI funcione
Crearla por API (setup por API)	✅ Rápido, confiable, autocontenido

Setup por API = usar la propia API para crear los datos que el test necesita como precondition. Lo implementamos en el fixture `createdBooking`:

```
createdBooking: async ({ bookingClient, authToken }, use) => {
  // setup: crear la reserva por API
  const payload = new BookingBuilder().build();
  const response = await bookingClient.create(payload);
  const id = (await response.json()).bookingid;

  await use({ id, payload }); // el test la recibe lista

  // teardown: borrarla al terminar (best-effort)
  await bookingClient.delete(id, authToken).catch(() => {});
},
```

Un test de lectura o update simplemente pide `createdBooking` y arranca con una reserva real garantizada:

```
test('GET /booking/{id} existente...', async ({ bookingClient, createdBooking }) => {
  const res = await bookingClient.getById(createdBooking.id); // ya existe
  // ...
});
```

Setup + teardown. Todo lo que va **antes** del `await use(...)` es preparación; lo que va **después** es limpieza. El `.catch(() => {})` en el delete es intencional: si el test ya borró la reserva (como el CRUD e2e) o el server la purgó, la limpieza no debe romper. Es un teardown "best-effort".

11. Datos con Builder: deep clone vs shallow clone

El `BookingBuilder` es el mismo patrón del Proyecto 1, pero tiene un detalle técnico importante en `build()`:

```
build(): Booking {  
  return structuredClone(this.booking); // copia PROFUNDA  
}
```

¿Por qué `structuredClone` y no `{ ...this.booking }`?

El spread (`{ ...obj }`) hace una **copia superficial (shallow)**: copia el primer nivel, pero los objetos **anidados** siguen siendo la **misma referencia** compartida. Nuestro `booking` tiene un objeto anidado (`bookingdates`). Con spread:

```
const a = builder.build();  
const b = builder.build();  
a.bookingdates.checkin = '2099-01-01'; // ⚠ con shallow clone, iesto también cambia  
b.bookingdates!
```

Como los tests corren **en paralelo**, dos tests podrían compartir y pisar el mismo objeto `bookingdates`, generando fallos aleatorios (flaky). `structuredClone` hace una **copia profunda (deep)**: clona también los objetos anidados, así cada `build()` devuelve datos 100% independientes.

12. Autenticación: token, Cookie y 403

Muchos endpoints (PUT, PATCH, DELETE) requieren autenticación. El flujo:

1. **Obtener un token:** `POST /auth` con usuario y contraseña devuelve `{ token: "abc123" }`.
2. **Usar el token:** enviarlo en el header `Cookie: token=abc123` en las requests protegidas.

En el `AuthClient`:

```
async getToken(username: string, password: string): Promise<string> {  
  const response = await this.createToken(username, password);  
  return (await response.json()).token;  
}
```

Y el fixture `authToken` lo obtiene una vez para el test que lo pida. Los tests protegidos lo reciben listo.

El caso negativo del 403. Probamos explícitamente qué pasa **sin** autenticación:

```
test('PUT sin token devuelve 403 Forbidden', async ({ request, createdBooking }) => {
  const response = await request.put(`/booking/${createdBooking.id}`, {
    data: new BookingBuilder().build(), // sin el header Cookie
  });
  expect(response.status()).toBe(403);
});
```

Verificar que un endpoint protegido **rechaza** a quien no está autenticado es tan importante como verificar que **acepta** a quien sí lo está. Es seguridad básica: si esto fallara (si dejara actualizar sin token), sería un agujero grave.

13. Métodos HTTP, códigos de estado e idempotencia

Repaso conceptual que este proyecto ejercita en la práctica:

Métodos

Método	Qué hace	¿Idempotente?
GET	Lee, no modifica nada	Sí
POST	Crea un recurso nuevo	No
PUT	Reemplaza el recurso completo	Sí
PATCH	Modifica parcialmente	Depende
DELETE	Elimina	Sí

Idempotente significa que ejecutar la operación una vez o muchas veces da el **mismo resultado**. **GET** no cambia nada. **PUT** deja el recurso en el mismo estado final sin importar cuántas veces lo mandes. **POST** **no** es idempotente: llamarlo dos veces crea **dos** recursos. Entender esto es clave para diseñar tests y para razonar sobre reintentos.

PUT vs PATCH (lo probamos en `update.spec.ts`)

- **PUT** reemplaza todo: mandás el objeto completo. Si omitís un campo, se pierde.
- **PATCH** actualiza solo lo que enviás. Nuestro test lo verifica: mandamos solo `firstname` y confirmamos que el `lastname` original **quedó intacto**.

```
await bookingClient.partialUpdate(id, { firstname: 'Parcial' }, token);
// ...
expect(body.firstname).toBe('Parcial'); // lo que cambiamos
expect(body.lastname).toBe(createdBooking.payload.lastname); // lo que NO tocamos
```

Códigos de estado que usamos

- **200 OK** — éxito con respuesta.
- **201 Created** — creado (y en esta API, curiosamente, también el DELETE).
- **403 Forbidden** — autenticado pero sin permiso / sin token.
- **404 Not Found** — el recurso no existe.

14. Encadenamiento: el CRUD end-to-end

El test estrella (`crud-e2e.spec.ts`) recorre el **ciclo de vida completo** de una reserva, donde el resultado de cada paso alimenta al siguiente. Eso es **encadenamiento (chaining)**:

```
// CREATE → obtenemos el id de la respuesta
const { bookingid } = CreateBookingResponseSchema.parse(await createRes.json());

// READ → usamos ese id, y verificamos que persistió idéntica a lo enviado
expect(BookingSchema.parse(await readRes.json())).toEqual(original);

// UPDATE → con token, cambiamos datos
// DELETE → borramos (i verifica 201!)
// VERIFY → GET del mismo id ahora da 404
expect(verifyRes.status()).toBe(404);
```

Por qué este test es valioso:

- Verifica que las operaciones **funcionan juntas**, no solo aisladas. Una API puede "crear" y "leer" bien por separado pero fallar en que lo creado sea realmente recuperable.
- El paso final (verificar 404 después de borrar) confirma que el DELETE **realmente eliminó** el recurso, no que solo devolvió un código lindo. Verificar el **efecto**, no solo la respuesta, es mentalidad de QA senior.
- Es autocontenido: crea y destruye su propia reserva, sin dejar rastro.

Este único test ejercita los 4 verbos del CRUD, la autenticación, la validación de contrato en cada paso y el encadenamiento. Es el que mejor "cuenta la historia" de la API.

15. Casos negativos: probar lo que NO debe pasar

Un error común es probar solo el "camino feliz" (todo sale bien). Los **casos negativos** —probar qué pasa cuando algo sale mal— son igual de importantes, y a menudo más. En este proyecto:

Test	Qué verifica
Auth con password incorrecta	Devuelve "Bad credentials" (status 200, la rareza)
GET de id inexistente	Devuelve 404
PUT sin token	Devuelve 403

Por qué importan: los bugs graves suelen esconderse en los caminos negativos. Que la app funcione cuando todo está bien es lo mínimo; que **falle de forma correcta y segura** cuando algo está mal (que no deje pasar a un usuario sin permiso, que no rompa ante un id inexistente) es lo que separa un sistema robusto de uno frágil.

16. Aislamiento en una API compartida

Restful-Booker es un **servidor compartido**: cualquiera en el mundo puede crear y borrar reservas ahí. ¿Cómo hacemos tests confiables sobre un backend que no controlamos?

Con aislamiento por datos propios. Cada test crea **su propia** reserva (vía `create` o el fixture `createdBooking`) y trabaja solo con **ese** id. Nunca dependemos de "la reserva número 5 que estaba ahí". Ejemplos:

- El test de lectura de un id existente crea su reserva y lee **esa**.
- El test de la lista (`GET /booking`) solo verifica que la lista **tiene elementos** (`length > 0`), no un id específico que otro podría borrar.
- El CRUD e2e crea, usa y borra su propia reserva.

Es el mismo principio del Proyecto 1 (tests como islas), aplicado a un backend compartido. **El aislamiento por datos propios es lo que permite correr en paralelo y de forma confiable** incluso contra un servidor que otros están usando al mismo tiempo.

17. La configuración de Playwright para API

`playwright.config.ts` tiene diferencias clave respecto al de UI:

```

use: {
  baseUrl: ENV.baseUrl, // rutas relativas en los clients
  extraHTTPHeaders: { // headers en TODAS las requests
    Accept: 'application/json',
    'Content-Type': 'application/json',
  },
  trace: 'on-first-retry',
},
// ⚠️ NO hay `projects` de navegadores.

```

Tres puntos:

1. **extraHTTPHeaders** : aplica estos headers a cada request automáticamente. El **Accept: application/json** es importante: sin él, algunas APIs (incluida esta) pueden responder en XML. Centralizarlo acá evita repetirlo en cada client.
2. **Sin projects de navegadores**. El testing de API no necesita Chromium/Firefox/WebKit. Por eso la config no define proyectos de browser y **no hace falta npx playwright install** .
3. **timeout generoso (30s)**. La API está en un hosting gratuito que a veces responde lento (arranque en frío). Damos margen para no tener falsos fallos por latencia de infraestructura.

18. CI/CD para API (más liviano)

El workflow ([.github/workflows/ci.yml](#)) es notablemente **más simple** que el del Proyecto 1:

```

- run: npm ci
- run: npm run typecheck
- run: npm test # ← sin 'npx playwright install' antes

```

No hay paso de instalación de navegadores porque no se usan. Menos pasos, ejecución más rápida, menos que puede fallar. Es otra ventaja concreta de la capa de API: **el CI de API es más barato y veloz que el de UI**.

Igual que en el Proyecto 1, corre en cada push y PR, verifica tipos antes de los tests, y publica el reporte como artifact.

19. Extensiones sugeridas

1. **Nuevo caso negativo**: creá un test que haga **POST /booking** con un body inválido (por ejemplo, **totalprice** como texto) y observá qué status devuelve la API. ¿Es lo que esperabas?

2. **Schema más estricto:** agregá `.strict()` a `BookingSchema` (que rechaza campos extra). Corré los tests. ¿Siguen pasando? ¿Qué implica para el contract testing?
3. **Nuevo endpoint:** implementá en un client el `GET /booking?firstname=X&lastname=Y` (filtro por nombre, que la API soporta) y testéalo.
4. **Idempotencia:** escribí un test que haga el mismo `PUT` dos veces y verifique que el resultado es idéntico (comprobando idempotencia).
5. **Encadenamiento propio:** creá dos reservas, listalas y verificá que ambos ids aparecen en la lista.
6. **Autenticación por Basic:** Restful-Booker también acepta auth por `Authorization: Basic`. Investigá y agregá una variante del client que use ese método.

20. Glosario

- **API:** interfaz por la que se comunican sistemas vía HTTP (request/response).
- **Endpoint:** una URL + método que expone una operación (ej: `POST /booking`).
- **Contract testing:** verificar la **forma** de la respuesta (campos y tipos), no solo el status.
- **Schema:** definición de la forma esperada de un dato. Aquí, con Zod.
- **Zod:** librería de validación de schemas en runtime para TS, que además genera tipos.
- **API Client:** clase que encapsula cómo se llama a un grupo de endpoints (el "POM" de API).
- **Setup por API:** crear los datos de precondition usando la propia API (rápido y aislado).
- **Encadenamiento (chaining):** usar la salida de una request como entrada de la siguiente.
- **Idempotente:** operación que da el mismo resultado ejecutada una o muchas veces.
- **Caso negativo:** prueba de qué pasa cuando algo sale mal (auth inválida, 404, 403).
- **Token:** credencial que se obtiene al autenticarse y se envía en requests protegidas.
- **Status code:** código HTTP de la respuesta (200, 201, 403, 404...).
- **Deep clone / shallow clone:** copia profunda (clona anidados) vs superficial (comparte anidados).
- **APIRequestContext:** el cliente HTTP de Playwright (fixture `request`).

21. Próximos pasos

Con los Proyectos 1 (UI) y 2 (API) ya cubrís las **dos capas principales de la pirámide de testing** con el mismo stack, de forma coherente. Lo que sigue en el roadmap:

- **Proyecto 3 — CI/CD completo:** llevar los pipelines (que ya existen en ambos repos) al siguiente nivel: smoke bloqueante vs regresión nightly, matriz, badges, notificaciones, y quizás correr UI + API juntos.
- **Proyecto 4 — Estabilidad y flakiness:** crear inestabilidad a propósito, medirla y eliminarla.

- **Proyecto 5 — Visual regression + contract testing con Pact:** el contract testing "consumer-driven" (Pact) lleva la idea de contratos de la sección 5 un paso más allá, verificándolos entre servicios.